INSTITUTO TECNOLÓGICO DE TLÁHUAC III

INGENIERÍA EN CIENCIAS DE DATOS

5to semestre

INTELIGENCIA ARTIFICIAL

Tarea 4

Comparativo de estrategias de búsqueda y su complejidad

DOCENTE: JOSÉ ISRAEL CAMPERO DOMÍNGUEZ

Alumna : Rico Zavaleta Astrid Paulina

Num. de Control : 241120408

FECHA DE ENTREGA: Viernes 09 - Octubre - 2026

Cuadro comparativo de los 10 algoritmos

Algoritmo	¿Completo?	¿Óptimo?	Tiempo	Espacio
BFS (anchura)	Sí	Con costos iguales	$O(b^{d+1})$	$O(b^{d+1})$
UCS (costo uniforme)	Sí, con costos positivos acotados inferiormente	Sí	$O(b^{1+[C^*/\varepsilon]})$	Igual al tiempo
DFS (profundidad)	No en general	No	$O(b^m)$	$O(bm)$
DLS (profundidad limitada)	Si el límite alcanza la solución, bajo condiciones finitas	No	$O(b^\ell)$	$O(b\ell)$
IDS (profundidad iterativa)	Sí	Con costos iguales	$O(b^d)$	$O(bd)$
Bidireccional (BFS en ambos sentidos)	Sí, bajo condiciones apropiadas	Con costos iguales	$O(b^{d/2})$	$O(b^{d/2})$
Voraz	No en general	No	$O(b^m)$ en el peor caso	$O(b^m)$
A*	Sí, bajo condiciones apropiadas	Sí, con heurística admisible; en búsqueda en grafo, consistente para la versión estándar	Exponencial en el peor caso	Exponencial en el peor caso
IDA*	Sí, bajo condiciones apropiadas	Sí, con heurística admisible y costos positivos	Exponencial en el peor caso	$O(bd)$ aproximadamente, para profundidad de solución d
RBFS	Sí, bajo condiciones apropiadas	Sí, con heurística admisible	Exponencial en el peor caso	$O(bd)$

Notación: b = factor de ramificación; d = profundidad de la solución menos profunda; m = profundidad máxima; ℓ = límite de profundidad; C* = costo óptimo; ε = costo mínimo positivo de una acción. Las cotas pueden variar según la implementación y los supuestos del problema.

NOTA: Se anexará al final el ejemplo resuelto a mano de la comparación de BFS, UCS y A

Optimalidad de A* con una heurística consistente

El algoritmo A* es una estrategia de búsqueda informada que selecciona el nodo con menor valor de la función:

$$f(n) = g(n) + h(n)$$

Donde:

- **g(n):** costo acumulado desde el estado inicial hasta el nodo actual.

- **h(n):** estimación del costo restante hasta alcanzar el objetivo.

- **f(n):** estimación del costo total de la solución que pasa por ese nodo.

Una heurística es **consistente** cuando cumple la siguiente desigualdad para cada transición entre dos nodos:

$$h(n) \leq c(n, n') + h(n')$$

Esto significa que la estimación desde un nodo no puede superar el costo de desplazarse a un vecino más la estimación desde ese vecino. Además, se establece h(objetivo) = 0.

Resumen de la demostración de optimalidad

1. La consistencia garantiza que los valores de f(n) no disminuyan a lo largo de un camino.

2. Por esta propiedad, cuando A* extrae un nodo de su frontera para expandirlo, ya conoce el camino de menor costo hacia ese nodo, suponiendo las condiciones habituales de búsqueda en grafo.

3. Si existiera una solución más barata que la primera meta extraída, habría algún nodo pendiente en ese camino con un valor f(n) menor o igual que el costo de esa solución.

4. Como A* siempre selecciona el menor valor de f(n), tendría que explorar ese nodo antes de seleccionar una meta más costosa, lo cual produce una contradicción.

Por lo tanto, A *con una heurística consistente garantiza encontrar una solución óptima*, siempre que exista una solución y se cumplan las condiciones apropiadas de la búsqueda.

Construcción de heurísticas mediante problemas relajados

Una heurística puede construirse simplificando el problema original, mediante la eliminación de algunas de sus restricciones. A esta simplificación se le conoce como **problema relajado**.

El costo óptimo de resolver el problema relajado puede utilizarse como estimación del costo restante en el problema original. Debido a que el problema relajado permite más acciones o impone menos restricciones, su solución no puede ser más costosa que la del problema original. Por ello, esta técnica permite construir heurísticas admisibles.

Ejemplo: Búsqueda de rutas

Supongamos que un vehículo debe trasladarse de una ciudad a otra utilizando una red de carreteras.

En el problema original, el vehículo solamente puede circular por las carreteras disponibles. En un problema relajado, eliminamos esa restricción y permitimos desplazarse directamente entre las ciudades.

La distancia en línea recta constituye una estimación del recorrido real por carretera. Si los costos representan distancias físicas, esta estimación no supera la distancia mínima por carretera.

Ventaja: Las heurísticas obtenidas mediante problemas relajados permiten orientar la búsqueda hacia el objetivo sin explorar todas las posibilidades.

Construcción de heurísticas mediante bases de datos de patrones

Las **bases de datos de patrones**, conocidas como *Pattern Databases* (PDB), son estructuras que almacenan previamente los costos óptimos de resolución de versiones simplificadas de un problema.

Para construirlas, se selecciona un subconjunto de los elementos del problema y se calculan las distancias mínimas desde los estados abstractos hasta el objetivo. Posteriormente, estos valores se almacenan en una tabla para consultarlos durante la búsqueda.

Ejemplo: Rompecabezas de ocho piezas

En el problema del 8-puzzle, el objetivo consiste en acomodar las fichas en un orden específico mediante movimientos permitidos.

Una base de datos de patrones puede considerar únicamente algunas fichas y omitir la posición exacta de las demás. Para cada configuración abstracta se almacena el número mínimo de movimientos necesarios para alcanzar el patrón objetivo.

Durante la ejecución de A*, el algoritmo consulta la base de datos para estimar el costo restante desde el estado actual.

Ventajas: Permite utilizar estimaciones más informativas que algunas heurísticas simples y reducir la cantidad de nodos explorados.

Limitación: Construir y almacenar estas bases puede requerir cantidades importantes de memoria y procesamiento.

En conclusión, tanto los problemas relajados como las bases de datos de patrones permiten diseñar heurísticas que mejoran la eficiencia de algoritmos de búsqueda informada como A*.

Conclusiones

El análisis comparativo permitió identificar que los algoritmos de búsqueda presentan diferencias importantes en completitud, optimalidad y consumo de recursos. BFS y UCS garantizan soluciones óptimas bajo determinadas condiciones, mientras que DFS puede utilizar menos memoria, pero no asegura encontrar la mejor solución.

El ejercicio resuelto mostró que la ruta con menor cantidad de pasos no necesariamente tiene el menor costo. Por ello, UCS y A* consideran los costos acumulados, mientras que A* incorpora adicionalmente una heurística para orientar la exploración.

Asimismo, se comprobó teóricamente que una heurística consistente permite garantizar la optimalidad de A* en búsqueda en grafo. Los problemas relajados y las bases de datos de patrones constituyen métodos útiles para construir estimaciones que reduzcan el esfuerzo de búsqueda.

En conclusión, la elección de un algoritmo depende de las características del problema, los recursos disponibles y la necesidad de obtener una solución óptima.

Referencias bibliográficas

Culberson, J. C., & Schaeffer, J. (1998). Pattern databases. *Computational Intelligence*, 14(3), 318–334. https://doi.org/10.1111/0824-7935.00065

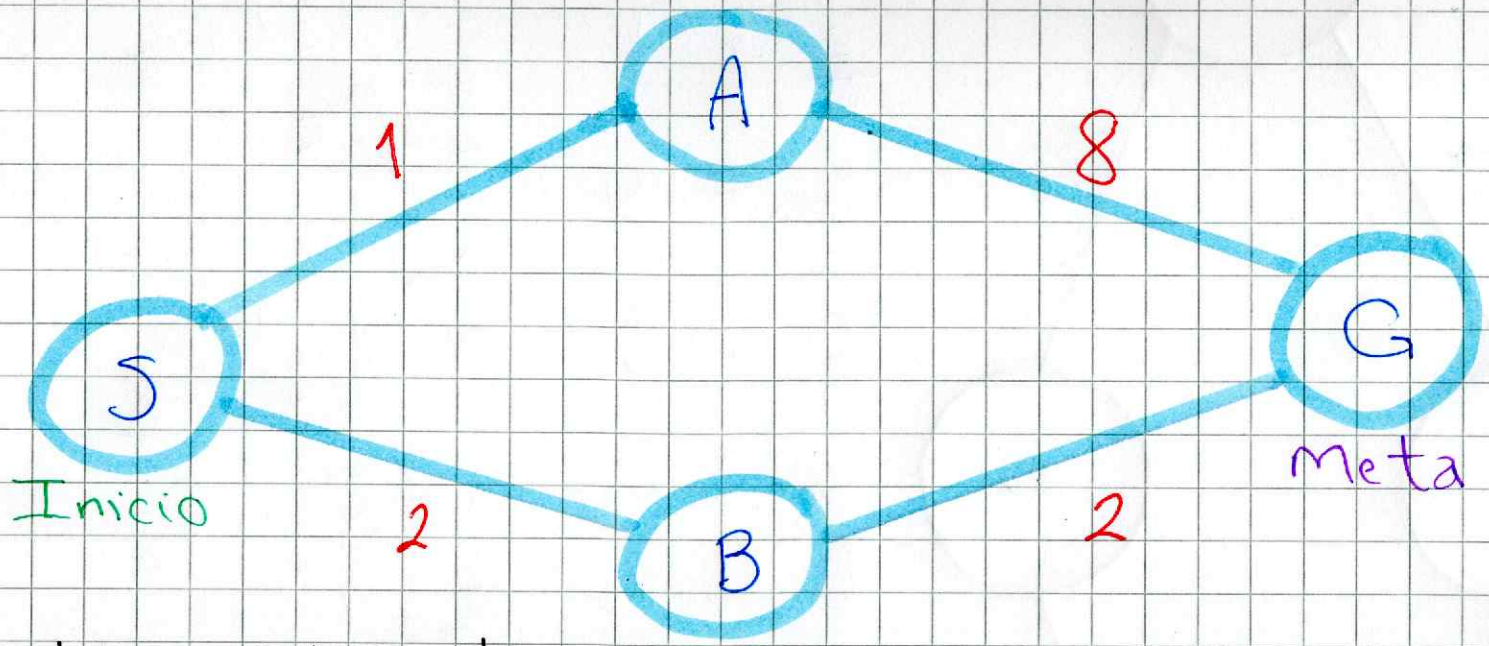
Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107. https://doi.org/10.1109/TSSC.1968.300136

Korf, R. E. (1985). Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence*, 27(1), 97–109. https://doi.org/10.1016/0004-3702(85)90084-0

Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

A continuación el ejemplo *resuelto a mano*: Comparación de BFS, UCS y A*

TAREA: Ejemplo a mano



Datos del problema

Estado inicial : S

Estado objetivo : G

Costos: S — A = 1 , S - B = 2

A - G = 8 , B - G = 2

Rutas posibles: S → A → G : costo 1+8 = 9

S → B → G : costo 2+2 = 4

Resolución mediante BFS

BFS explora por niveles, sin considerar los costos de las aristas.

Suponiendo que se visita A antes que B:

1. Se expande S y se descubren A y B.
2. Se expande A y se descubre G.
3. Se expande B, sin encontrar una ruta más corta en número de pasos.
4. Se extrae G de la cola y termina la búsqueda.

Resultado: S → A → G,
costo 9,
profundidad 2.

Resolución mediante UCS

UCS selecciona siempre el nodo pendiente con menor costo acumulado $g(n)$

Paso	Nodo expandido	Costos relevantes
1	S	A = 1 , B = 2
2	A	G_1 = 9
3	B	G_1 se actualiza a 4
4	G_1	Se alcanza la meta

Resultado: S → B → G_1 ,

costo 4

Resolución mediante A*

Utilizaremos esta heurística:

Nodo	h(n)
S	4
A	3
B	2
G	0

La función de evaluación es:

$$f(n) = g(n) + h(n)$$

Paso	Nodo	g(n)	h(n)	f(n)
1	S	0	4	4
2	B	2	2	4
3	G	4	0	4

Al expandir S,
A tiene $f(A) = 1 + 3 = 4$
y B tiene $f(B) = 2 + 2 = 4$
Existe un empate; para este ejemplo elegí B primero.
Después, G_1 tiene $f(G_1) = 4$, mientras A sigue con $f(A) = 4$; nuevamente elegimos G_1 primero.
Resultado: $S \to B \to G_1$, costo 4.

Esta heurística es consistente porque cumple $h(u) \leq c(u, v) + h(v)$ en cada arista del grafo.